Boosting (All below info learnt from Campusx and Krish Naik)

Boosting is combining weak models to create one single strong model where each model learns from where its previous model has made errors, so it can be careful on those values especially.
There are 3 types of boosting : Adaboost, Gradient Boosting, XGBoost and Catboost. We'll talk about Gradient Boosting here first. Gradient Boosting uses Decision Tree internally as its each model.

Gradient Boosting on Training Dataset

Gradient Boosting, where each model (except the first) is trained(obviously with input columns but the output column is the Error(actual values - predicted values) of the previous model) and predicts the Error the previous model has made.
Each model predicts less error converging towards almost 0 error slowly.

Base Model → 1st Tree Model → 2nd Tree Model …..

Assume n_estimators = 2 i.e. there are 2 Tree models in GradientBoosting and the Input dataset :

iq	cg	salary($)
20	3.7	52
10	3.3	33
33	3.0	40
35	3.2	35
36	3.5	38
37	3.5	45
40	3.5	45
38	3.7	46
32	3.4	44

The **base** model :

Input columns		Output column	Base Model	
iq	cg	salary($)	pred_salary0	res0
20	3.7	52	42	10
10	3.3	33	42	- 9
33	3.0	40	42	- 2
35	3.2	35	42	- 7
36	3.5	38	42	- 4
37	3.5	45	42	3
40	3.5	45	42	3
38	3.7	46	42	4
32	3.4	44	42	2

Base Model is actually not a model but just some straight calculations.

[pred = Prediction, res = **Residual Error (Actual Salary - Predicted Salary)**.]
The Base Model, doesn't predict anything and just quantifies the Mean of Output column (salary) as prediction alongside the residual error.

The **first** model : (**Decision Tree model is used with max_depth = 8 to 32**)

Input columns		Output column	Base Model		Model_1	
iq	cg	salary($)	pred_salary0	res0	pred_res1	res1
20	3.7	52	42	10	10	9.0
10	3.3	33	42	- 9	-9	-8.1
33	3.0	40	42	- 2	-2	-1.8
35	3.2	35	42	- 7	-7	-6.3
36	3.5	38	42	- 4	-4	-3.6
37	3.5	45	42	3	3	2.7
40	3.5	45	42	3	3	2.7
38	3.7	46	42	4	4	3.6
32	3.4	44	42	2	2	1.8

pred_res1 :
The first model(decision tree) predicts the error of its previous | base model, (**res0**), because we train the first model with input columns (iq, cg) and output column (res0, not “salary”). Why did the 1st model predictc (pred_res1) the exact same error of the base model (res0)? Because our dataset is tiny and with max_depth = 8, it only needs a maximum 3rd depth level to construct the whole full grown tree. So it could predict perfectly.

res1 :
If the first model predicts “error”, not the actual output column, salary. Then how can we use this decision tree model to calculate the salary that it would predict if we had given it the actual output column, salary. not ‘res0’?

For the 1st row (iq = 20, cg = 3.7), pred_salary1_1st_row
= pred_salary0_1st_row + pred_res1_1st_row
(**summation of whatever So Far models predicted**)
= 42 + 10
= 52. And the actual salary is also 52 which is a clear sign of overfitting, low bias and high variance. So we multiply learning_rate(0.1) :

For the 1st row (iq = 20, cg = 3.7), pred_salary1_1st_row
= pred_salary0_1st_row + (learning_rate * pred_res1_1st_row)
(**summation of whatever So Far models predicted with learning_rate being multiplied for 1st to last model**)
= 42 + (0.1 * 10)
= 43, overfitting issue solved.

So, **pred_salary1** = pred_salary0 + (learning_rate * pred_res1)

Now, res1 (residual error the 1st model made)
= Actual Value - Predicted Value
= Actual Salary - Predicted Salary 1st Model can make
= salary($) - pred_salary1

E.g., for the first row, res1 = 52 - {42 + (0.1 * 10)} = 52 - 43 = 9, which we can verify in the above picture.

The **second** model :

Input columns		Output column	Base Model		Model_1		Model_2	
iq	cg	salary($)	pred_salary0	res0	pred_res1	res1	pred_res2	res2
20	3.7	52	42	10	10	9.0	9.0	8.10
10	3.3	33	42	- 9	-9	-8.1	-8.1	-7.29
33	3.0	40	42	- 2	-2	-1.8	-1.8	-1.62
35	3.2	35	42	- 7	-7	-6.3	-6.3	-5.67
36	3.5	38	42	- 4	-4	-3.6	-3.6	-3.24
37	3.5	45	42	3	3	2.7	2.7	2.43
40	3.5	45	42	3	3	2.7	2.7	2.43
38	3.7	46	42	4	4	3.6	3.6	3.24
32	3.4	44	42	2	2	1.8	1.8	1.62

Same as the first model.

pred_res2 :
The second model(decision tree) predicts the error of its previous | 1st model, (**res1**), because we train the second model with input columns (iq, cg) and output column ("res1", since the 2nd model will predict the 1st model's error).

res2 :

pred_salary2 = pred_salary0 + (learning_rate * pred_res1) + (learning_rate * pred_res2)

(**summation of whatever So Far models predicted with learning_rate being multiplied for 1st to last model**)

Now, res2 (residual error the 2nd model made)
= Actual Value - Predicted Value
= Actual Salary - Predicted Salary 2nd Model can make
= salary($) - pred_salary2

E.g., for the first row, res2 = 52 - {42 + (0.1 * 10) + (0.1 * 9)} = 52 - 43.9 = 8.1, which we can verify in the above picture.

This pred_salary2 is the official output of the Gradient Boosting Regressor after 2 boosting iterations on the actual output(salary) for the training dataset(x_train, y_train).

Observations

---

1) m = model(decision tree) numbers), y = output column.

$$\text{predicted_y, } \hat{y} = \text{predicted_y_0} + \sum_{i=1}^{m} (\text{learning_rate} * \text{predicted_res_}i)$$

2) Each model in gradient boosting learns from its previous model's error.

Code

```
def GBR_fit(X: pd.DataFrame, y: pd.Series, n_estimators: int, lr: float):
    df = pd.concat([X, y], axis=1)

    # For the Base Model which is not a Machine Learning Model but a Box of
calculations.
    df['pred_salary_0'] = df['salary($)'].mean()
    df['res_0'] = df['salary($)'] - df['pred_salary_0']
    pred_salary_for_current_model = df['pred_salary_0'].values # pred =
predicted.

    # For the Tree models.
    for i in range(1, n_estimators + 1):
        cur_tree = DecisionTreeRegressor(max_depth = 10, random_state =
42).fit(X, df[f'res_{i-1}'])
        df[f'pred_res_{i}'] = cur_tree.predict(X) # whatever tree predicted.
        pred_salary_for_current_model += (lr * df[f'pred_res_{i}']) #
sum_of_all_pred_salary.
        df[f'res_{i}'] = df['salary($)'] - pred_salary_for_current_model
        df[f"pred_salary_{i}"] = pred_salary_for_current_model

    return df
GBR_fit(X = df[['iq', 'cg']], y = df['salary($)'], n_estimators = 3, lr = 0.1)
```

Output :

	iq	cg	salary($)	pred_salary_0	res_0	pred_res_1	res_1	pred_salary_1	pred_res_2	res_2	pred_salary_2	pred_res_3	res_3	pred_salary_3
0	20	3.7	52	42.0	10.0	10.0	9.0	43.0	9.0	8.10	43.90	8.10	7.290	44.710
1	10	3.3	33	42.0	-9.0	-9.0	-8.1	41.1	-8.1	-7.29	40.29	-7.29	-6.561	39.561
2	33	3.0	40	42.0	-2.0	-2.0	-1.8	41.8	-1.8	-1.62	41.62	-1.62	-1.458	41.458
3	35	3.2	35	42.0	-7.0	-7.0	-6.3	41.3	-6.3	-5.67	40.67	-5.67	-5.103	40.103
4	36	3.5	38	42.0	-4.0	-4.0	-3.6	41.6	-3.6	-3.24	41.24	-3.24	-2.916	40.916
5	37	3.5	45	42.0	3.0	3.0	2.7	42.3	2.7	2.43	42.57	2.43	2.187	42.813
6	40	3.5	45	42.0	3.0	3.0	2.7	42.3	2.7	2.43	42.57	2.43	2.187	42.813
7	38	3.7	46	42.0	4.0	4.0	3.6	42.4	3.6	3.24	42.76	3.24	2.916	43.084
8	32	3.4	44	42.0	2.0	2.0	1.8	42.2	1.8	1.62	42.38	1.62	1.458	42.542

Salary = 52 → 42(Base) → 43.0 → 43.90 → 44.710 (From 42, converging towards actual salary 52 by increasing in each tree model)

Salary = 40 → 42(Base) → 41.8 → 41.62 → 41.458 (From 42, converging towards actual salary 40 by decreasing in each tree model)

Gradient Boosting on Testing Dataset

[My favorite AI, Deepseek, helped me to understand]

We've seen how Gradient Boosting Regressor predicts the output column, salary, on training dataset. But how does it predict for new data(iq, cg)? E.g., for cg = 3.9, iq = 10 what is the Predicted salary($)?

Wait! How did we predict for the training dataset for 2 tree models?

pred_salary2 = pred_salary0 + (learning_rate * pred_res1) + (learning_rate * pred_res2)

pred_salary0 = The Base model is actually No Model which returns the mean of the output column which is just a scalar value.

pred_res1 = We train the first tree model on (iq, cg, res0). Then we predict each training input (iq, cg) and for each training input we get residual_error.

pred_res2 = We train the second tree model on (iq, cg, res1). Then we predict each training input (iq, cg) and for each training input we get residual_error.

So each model is trained on the same input data and it returns|predicts Residual Error which we use to quantify Predicted Salary for that model.

Now, after training the GradientBoostingRegressor, to predict the salary for cg = 3.9, iq = 10, we send this data to each tree model (the base model's predicted_value is constant since its not a tree model) and each tree model returns the Residual Error as its prediction which is assume pred_res1 = 9.7, pred_res2 = -8 and pred_salary0 is 42 from the training.

So predicted_salary = pred_salary0 + (learning_rate * pred_res1) + (learning_rate * pred_res2)

= 42 + (0.1 * 9.7) + (0.1 * -8)

= 42.17 $

This is how prediction happens on unseen|testing data in Boosting. For each testing data : We give to each Machine Learning Model and each ML Model returns something(Residual Error or predicted_salary etc) from their prediction i.e. model.predict() and we add all the "something" in a specific way to get the final Predicted Output.

Math from scratch :

Why did the Base model quantify the Mean, not summation or any other equation? See "**Gradient Boosting Regression Math.pdf**" in the Theory folder.